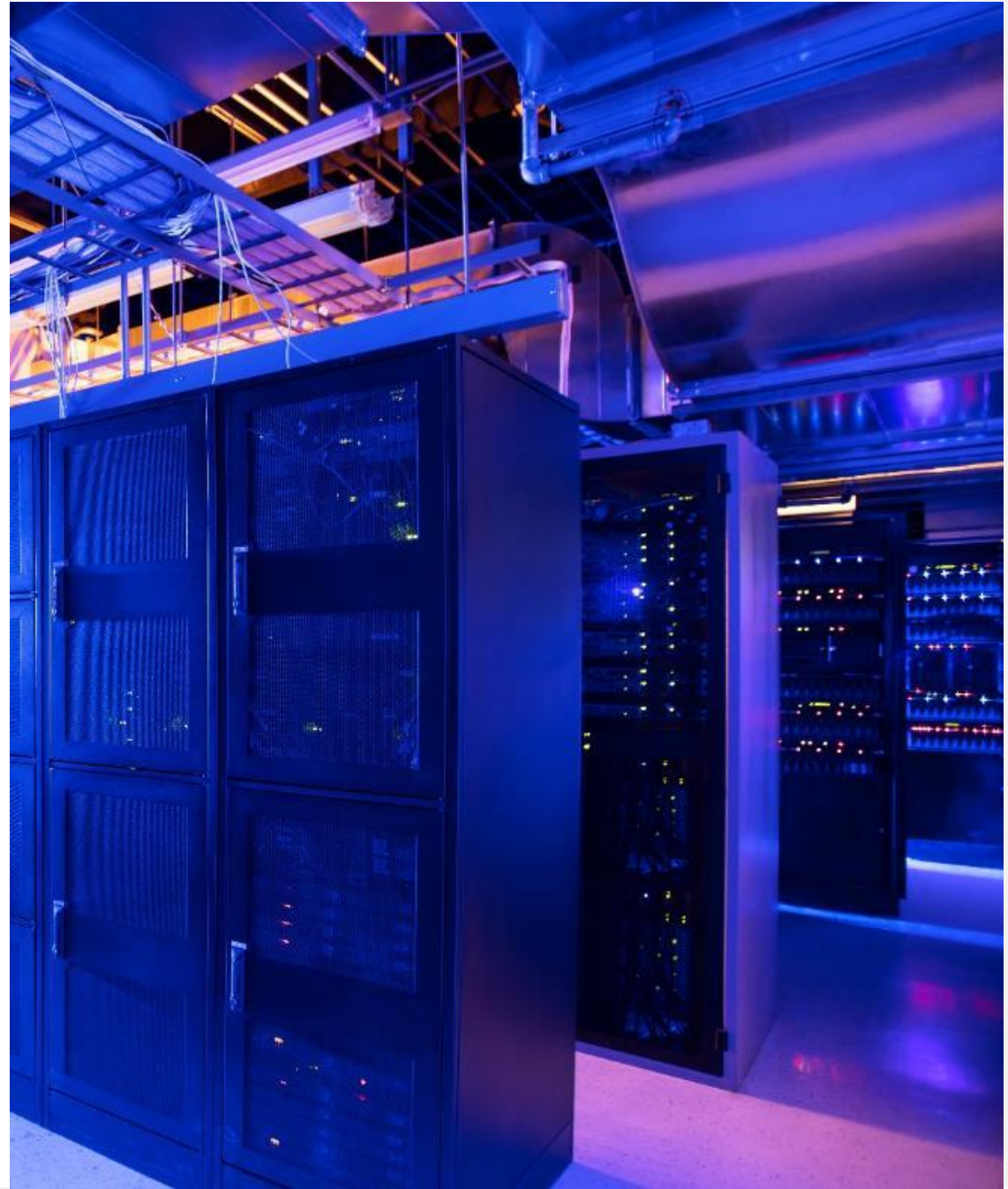


Netname space (Docker) with RXE

Zhu Yanjun Dec. 2023

AGENDA

- Docker & Net namespace
- RDMA & SoftRoCE
- SoftRoCE architecture
- Analyze the root cause
- How to fix it
- Demo



Docker

1. Introduction

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications.

2. Docker foundational layer

- 1) cgroup

- 2) chroot

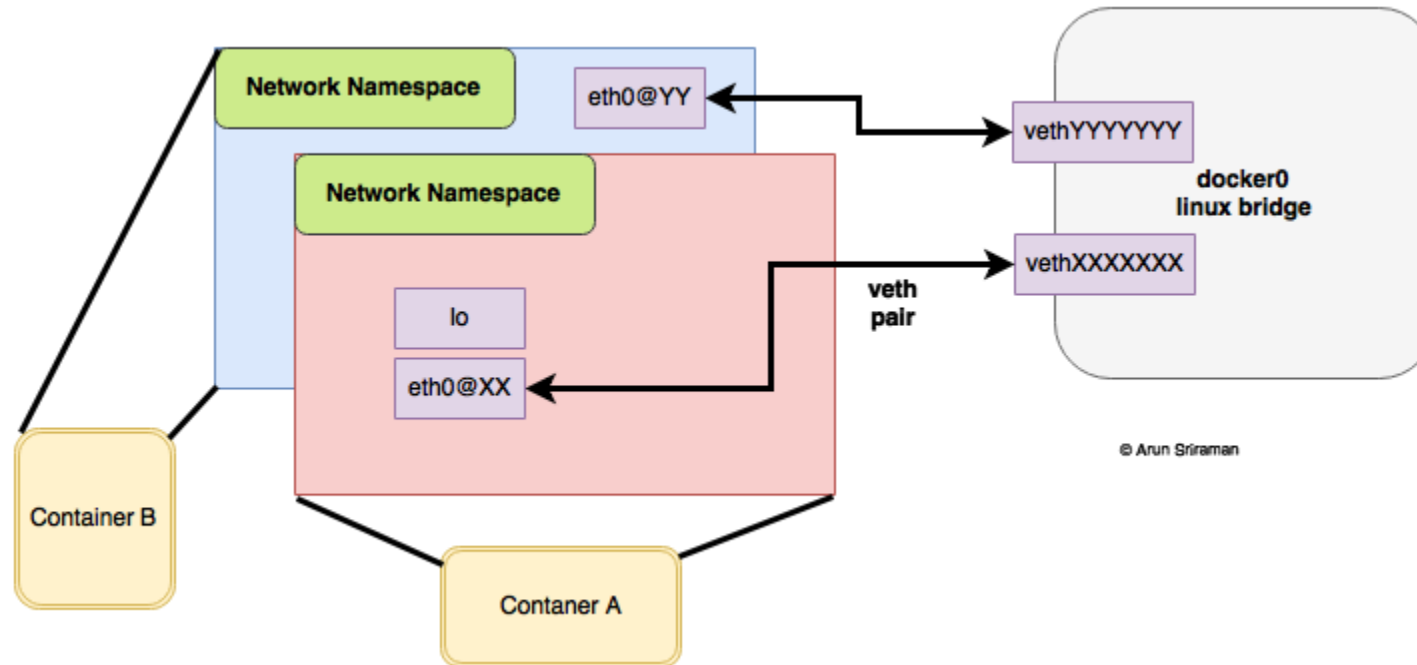
- 3) namespace

 - net namespace is to configure network in the container

Net namespace

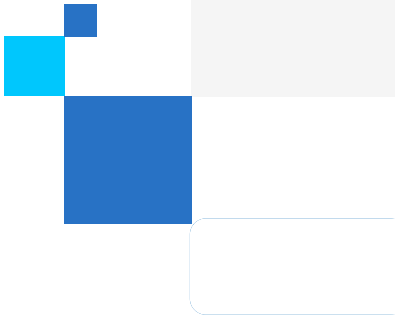
1. Net namespace

Linux network namespace technology is for network isolation, each docker container has its own network namespace, which means it has its own IP address, routing table, etc.



RDMA Introduction

1. RDMA is short for Remote Direct Memory Access.
It is A (relatively) new method for interconnecting platforms in high-speed networks that overcomes many of the difficulties encountered with traditional networks such as TCP/IP over Ethernet.
 - 1) new standards
 - 2) new protocols
 - 3) new hardware interface cards and switches
 - 4) new software
2. RDMA implementations
Infiniband and RoCEv1/v2



Infiniband and RoCE

1. Infiniband

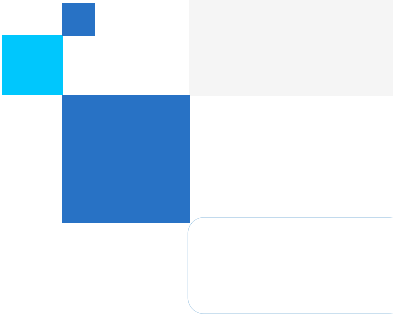
Infiniband implementation needs special devices, for example HCA (Host Channel Adapter), IB switch and special cables.

2. RoCE

1) RoCEv1 and RoCEv2

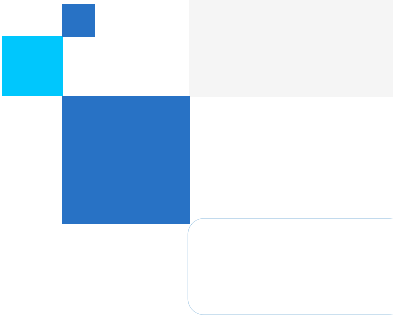
2) RoCEv1 is based on TCP/IP ethernet layer. It only works in LAN (Local Area Network). It is not popular.

3) RoCEv2 is based on UDP. It is popular in public/private Cloud and Data Center.



RoCEv2

1. RoCEv2 is based on UDP/IPv4 or UDP/IPv6. The target udp port 4791 is reserved for RoCEv2. Because RoCEv2 can be routable, RoCEv2 sometimes can be called Routable RoCE or RRoCE.
2. RoCEv2 is supported in Mellanox OFED2.3 or above. In Linux 4.5, RoCEv2 is supported. Intel E810 also supports RoCEv2.



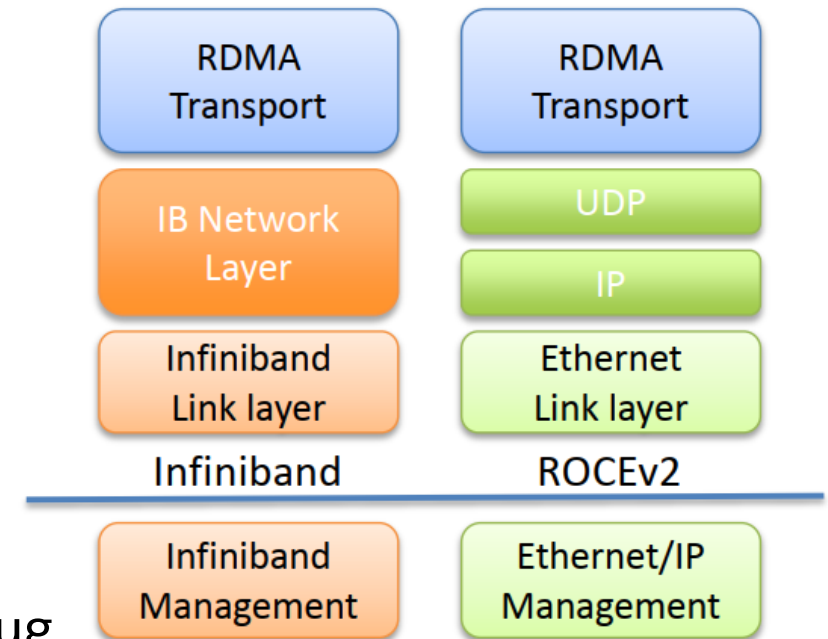
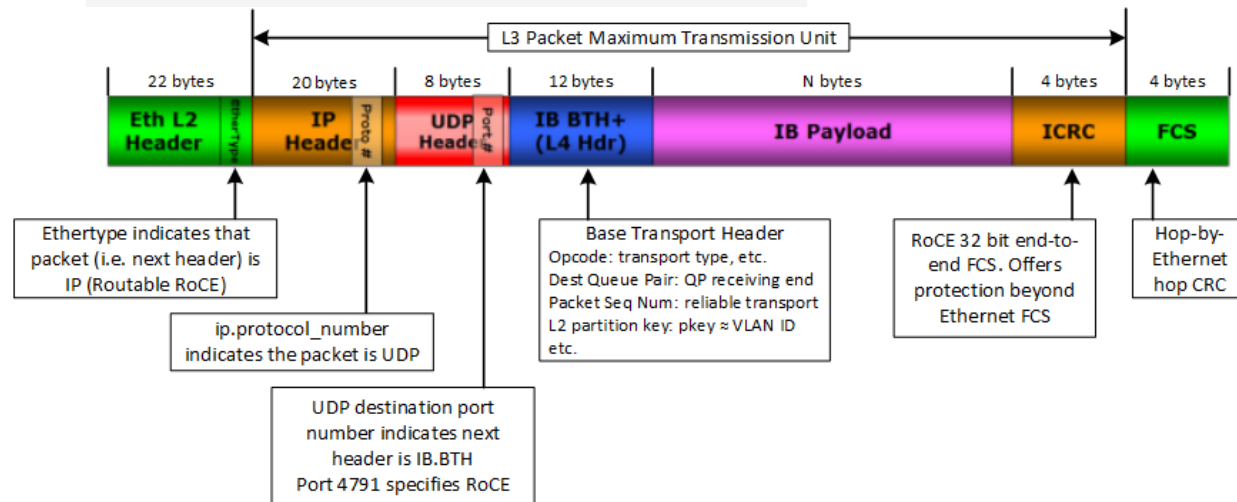
RDMA summary

1. RDMA is a networking performance optimization
Traditional networking stacks (sockets/TCP/IP/Ethernet) have inherent performance barriers and CPU overhead. RDMA is designed to reduce or eliminate these.

RDMA Techniques	Benefit		
	CPU Util	Latency	Mem bw
Offload network transport (e.g. UDP/IP) from Host	✓	✓	
Eliminate receive memory copies with tagged buffers	✓	✓	✓
Reduce context switching with OS bypass (map NIC hw resources into user space)	✓	✓	
Define an asynchronous “verbs” API (socket is synchronous)	✓		✓
Preserve message boundaries to enable application (e. g. SCSI) header/data separation	✓		✓
Message-level (not packet-level) interrupt coalescing	✓		

RDMA SoftRoCE

1. SoftRoCE is a software implementation of the IBTA RoCEv2 specification RDMA transport services over Ethernet network. The following is the packet format of RDMA. It is in UDP payload. It includes IB BTH, IB payload and ICRC. The left is the comparison between Infiniband and RoCEv2. Infiniband stack includes IB network, IB Link and IB management while RoCEv2 is in UDP payload, it works in TCP/IP stack.



2. Use cases

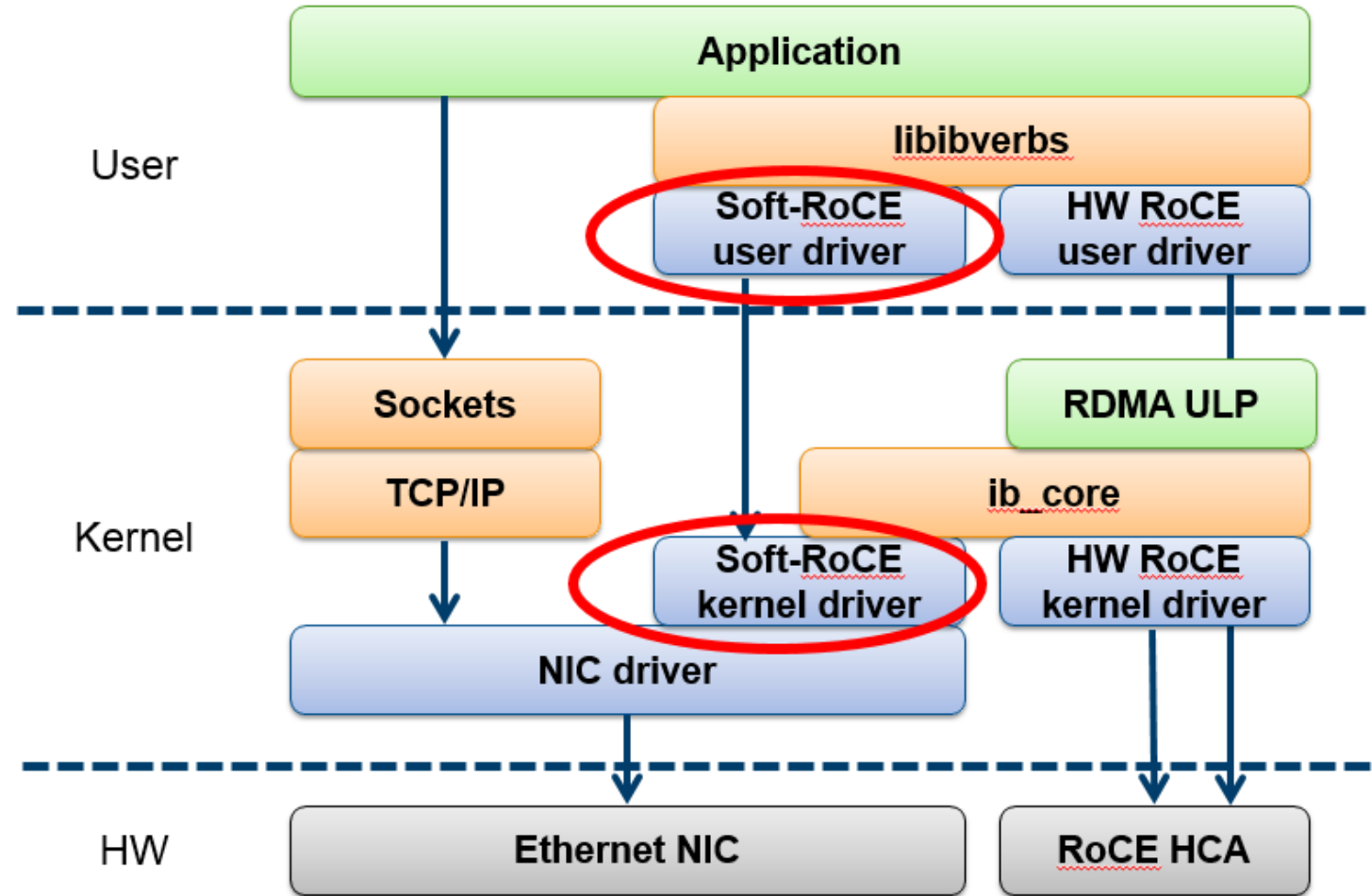
- 1) Develop new RDMA features on SoftRoCE
- 2) Develop application on SoftRoCE, easy to debug
- 3) Make function tests on SoftRoCE
- 4) Asymmetric deployments, SoftRoCE connect to RDMA hardware, e.g. MLX5, E810

RDMA SoftRoCE Architecture

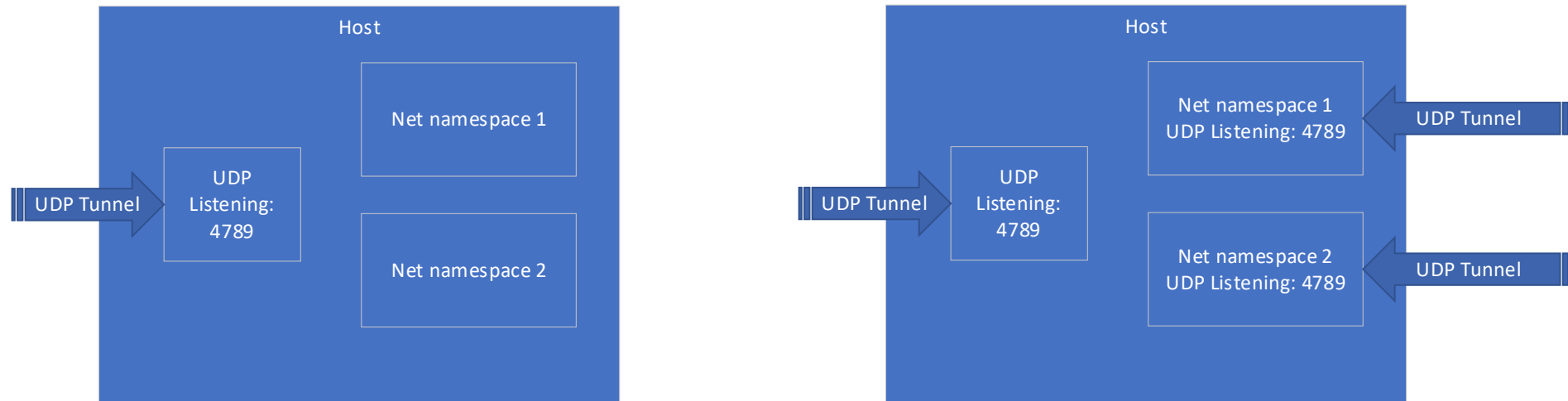
User Space: SoftRoCE user driver is in rdma-core software package. IB verbs generic APIs finally call the specific implementations of SoftRoCE user driver.

Kernel: SoftRoCE kernel driver is attached on a specific NIC driver(ICE driver). It receives rdma packets from RDMA stack, build a skb and put rdma data into UDP payload, then send skb to TCP/IP stack.

HW: Any NIC including E810 (Intel). In my demo, I use E810 without irdma, attach SoftRoCE on E810



Why SoftRoCE not work on Net namespace(Docker)



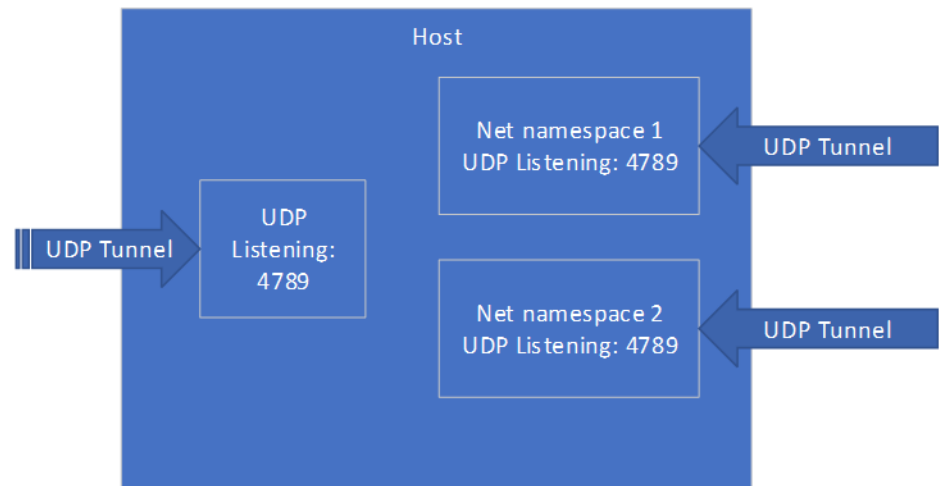
In the left, when RXE driver is loaded in the host, the udp port 4791 is listened on the Host. Because the separation of net namespace, net namespace 1 and 2 will not use this udp port 4791. So RXE can not be used in net namespace.

In the right, when RXE rdma link is created in the host, udp port 4791 is listened on the host. When RXE rdma link is created in net namespace, the udp port 4791 is listened in the net namespace. So RXE can be used in the host and net namespace.

The root cause is that udp port 4791 is created in the host when RXE driver is loaded.

How to fix it - 1

Solution: The problem is that the udp port 4791 is only listened in the host. The process in the net namespace can not access this udp port 4791 in the host. As such, the plan is to make udp port 4791 is listened in the host and every net namespace.



The above is the solution. That is, UDP port 4791 is listened when rdma link of rxe is created in the host or the net namespaces.

How to fix it - 2

Problem: When RXE is loaded with modprobe command, the UDP port 4791 is listened in the host. It is difficult to make the process in the net namespace be able to access the udp port 4791 in the host.

And currently, no way to create UDP port 4791 in net namespace.

Solution: Change the way to create the listened udp port 4791. Now when RXE is loaded, the udp port 4791 is not listened in the host. When “rdma link add ...” command is run, the udp port 4791 is created. When the above command is run in the host, this udp port 4791 is created in the host. When in net namespace, this udp port is created in the net namespace.

```
# modprobe -v rdma_rxe
insmod /lib/modules/6.5.5+/kernel/net/ipv4/udp_tunnel.ko
insmod /lib/modules/6.5.5+/kernel/net/ipv6/ip6_udp_tunnel.ko
insmod /lib/modules/6.5.5+/updates/drivers/infiniband/sw/rxe/rdma_rxe.ko
# ss -ltn | grep 4791
UNCONN 0      0          0.0.0.0:4791      0.0.0.0:*
UNCONN 0      0          [::]:4791        [::]:*
```

How to fix it – 2 – 1 host

In the net namespace, the screenshot as below, when rdma_rxe kernel module is loaded into linux kernel via modprobe command, the UDP port 4791 is not listened in the host.

After running the command “rdma link add ...” in the host, the UDP port 4791 is listened in the net namespace net0, and a rdma link of rxe is created.

```
# modprobe -v rdma_rxe
insmod /lib/modules/6.5.5+/kernel/net/ipv4/udp_tunnel.ko
insmod /lib/modules/6.5.5+/kernel/net/ipv6/ip6_udp_tunnel.ko
insmod /lib/modules/6.5.5+/updates/drivers/infiniband/sw/rxe/rdma_rxe.ko
# ss -ltn | grep 4791

# rdma link add rxe0 type rxe netdev eno1
# ss -ltn | grep 4791
UNCONN 0      0      0.0.0.0:4791      0.0.0.0:*
UNCONN 0      0      [::]:4791        [::]:*
```

How to fix it – 2 – 2 – net namespace

In the net namespace, the screenshot as below, when a new net namespace net0 is created, no UDP port 4791 is listened in the net namespace net0.

After running the command “rdma link add ...” in the net namespace net0, the UDP port 4791 is listened in the net namespace net0.

```
# ip netns add net0
# ip netns exec net0 ss -ltn | grep 4791
# ip netns exec net0 rdma link add rxe1 type rxe netdev lo
# ip netns exec net0 ss -ltn | grep 4791
UNCONN 0      0      0.0.0.0:4791      0.0.0.0:*
UNCONN 0      0      [::]:4791        [::]:*
```

How to fix it - 3

Problem: The command “rdma link del ...” will not disable listening UDP port 4791. The root cause is that delink does not exist.

```
struct rdma_link_ops {  
    struct list_head list;  
    const char *type;  
    int (*newlink)(const char *ibdev_name, struct net_device *ndev);  
};
```

Solution: Add delink in the struct, and implement rxe_del_link in the RXE

```
# rdma link add rxe0 type rxe netdev eno0  
# rdma link  
link rxe0/1 state ACTIVE physical_state LINK_UP netdev eno0  
# ss -lun | grep 4791  
UNCONN 0      0      0.0.0.0:4791      0.0.0.0:*  
UNCONN 0      0      [::]:4791        [::]:*  
# rdma link del rxe0  
# rdma link  
# ss -lun | grep 4791  
UNCONN 0      0      0.0.0.0:4791      0.0.0.0:*  
UNCONN 0      0      [::]:4791        [::]:*
```

How to fix it – 3 - 1

After add a function pointer delink in struct rdma_link_ops, and implement a rxe_delink function. After running “rdma link del ...”, the UDP port 4791 is not listened.

```
struct rdma_link_ops {  
    struct list_head list;  
    const char *type;  
    int (*newlink)(const char *ibdev_name, struct net_device *ndev);  
    int (*dellink)(struct ib_device *dev);  
};
```

```
# rdma link add rxe0 type rxe netdev eno1  
link rxe0/1 state ACTIVE physical_state LINK_UP netdev eno1  
# ss -ltn | grep 4791  
UNCONN 0      0          0.0.0.0:4791      0.0.0.0:*  
UNCONN 0      0          [::]:4791        [::]:*  
# rdma link del rxe0  
# ss -ltn | grep 4791
```

How to fix it - 4

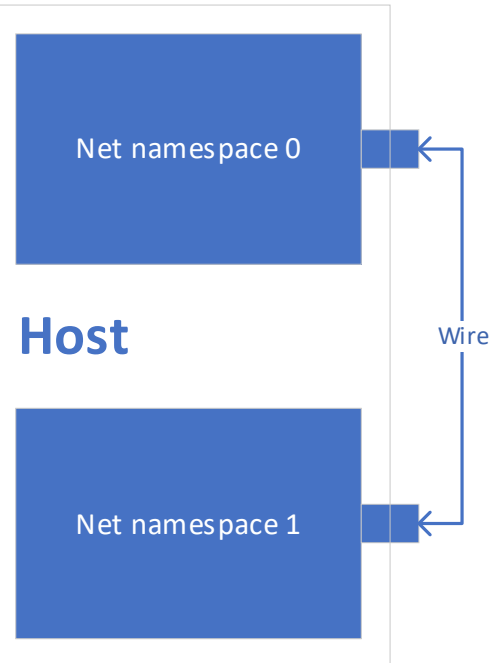
Problem: When rdma links of rxe are created in host and net namespace, how to save these contexts? As such, these contexts can be fetched easily and conveniently?

Solution: 2 files rxe_ns.c and rxe_ns.h are created, struct pernet_operations is initialized. When “rdma link add ...” command is run, the context will be saved into this struct based on net namespace id. When the context information is needed, the net namespace information is passed to struct pernet_operations, then the net namespace context information can be fetched.

The following is the core struct. The host and net namespaces store the contexts in this struct.

```
/*
 * Per network namespace data
 */
struct rxe_ns_sock {
    struct sock __rcu *rxe_sk4;
    struct sock __rcu *rxe_sk6;
};
```

Support of net namespace - Topology



```
# ip netns exec net0 ip 1
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
3: eno2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 00:1e:67:3e:48:7f brd ff:ff:ff:ff:ff:ff
# ip netns exec net1 ip 1
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4: eno3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 00:1e:67:3e:48:80 brd ff:ff:ff:ff:ff:ff
# ip netns exec net0 rping -s -a 192.168.1.2 -v -C 2 > /dev/NULL &
[1] 14310
# ip netns exec net1 rping -c -a 192.168.1.2 -v -C 2
ping data: rdma-ping-0: ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz
ping data: rdma-ping-1: BCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz
server DISCONNECT EVENT...
wait for RDMA_READ_ADV state 10
client DISCONNECT EVENT...
```

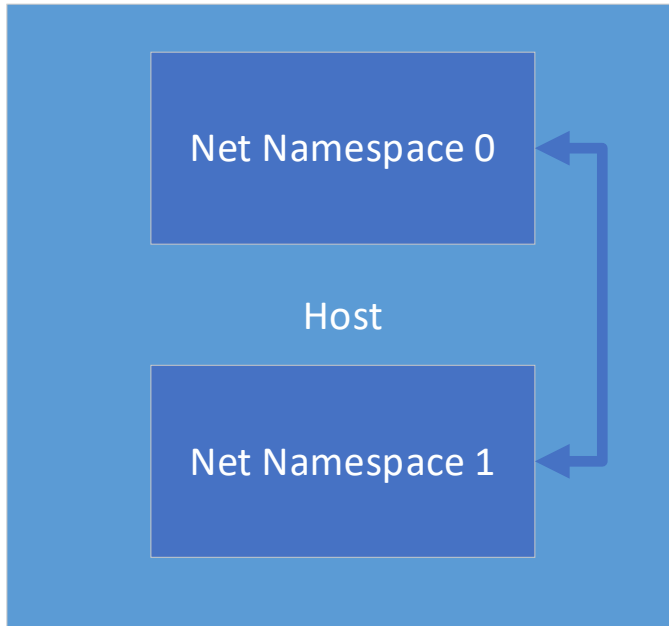
RDMA/rxe: Replace l_sk6 with sk6 in net namespace
RDMA/rxe: Add the support of net namespace notifier
RDMA/rxe: add the support of net namespace
RDMA/rxe: Replace global variable with sock lookup functions
RDMA/rxe: Implement dellink in rxe
RDMA/nldev: Add dellink function pointer
RDMA/rxe: Support more rdma links in init_net
RDMA/rxe: Creating listening sock in newlink function

1.Scenario: net namespace connect to the host via veth



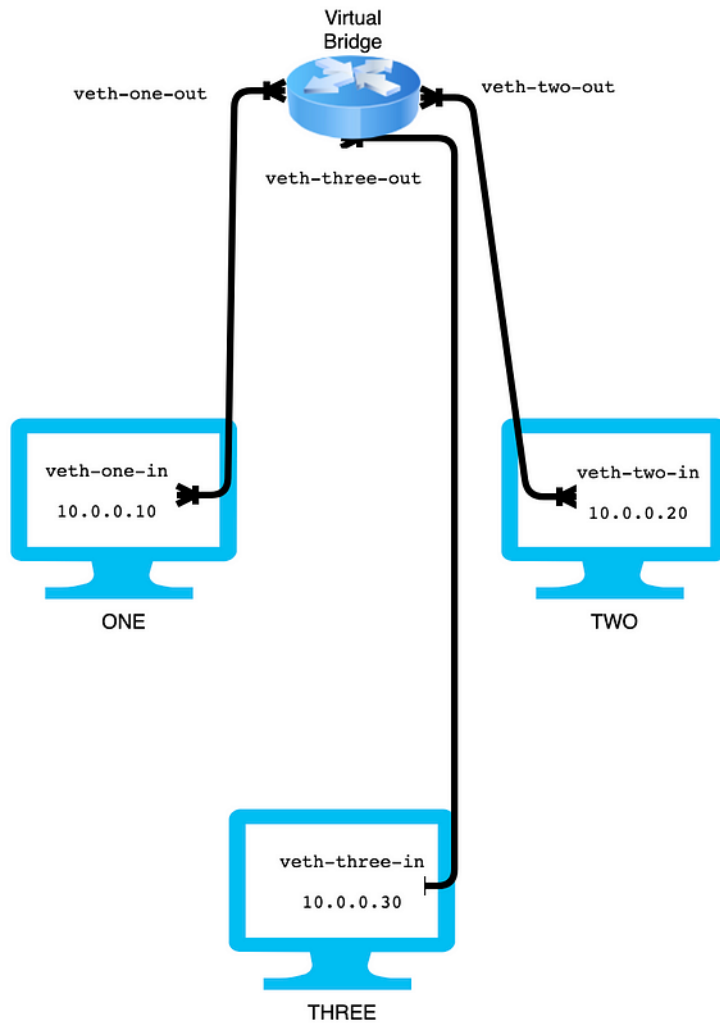
```
ip netns add test1
ip netns ls
ip link add veth-a type veth peer name veth-b
ip link set veth-a netns test1
ip netns exec test1 ip l set veth-a up
ip netns exec test1 ip addr add 1.1.1.1/24 dev veth-a
ip netns exec test1 ip -4 a
ip netns exec test1 rdma link add rxe0 type rxe netdev veth-a
ip netns exec test1 rping -s -a 1.1.1.1&
ip link set veth-b up
ip addr add 1.1.1.2/24 dev veth-b
ping -c 3 1.1.1.1
rdma link add rxe1 type rxe netdev veth-b
rping -c -a 1.1.1.1 -d -v -C 3
ip netns del test1
```

2.Scenario: 2 net namespaces connected via veth



```
ip netns add test1
ip link add veth-a type veth peer name veth-b
ip link set veth-a netns test1
ip netns exec test1 ip l set veth-a up
ip netns exec test1 ip addr add 1.1.1.1/24 dev veth-a
ip netns exec test1 rdma link add rxe0 type rxe netdev veth-a
ip netns exec test1 rdma link
ip netns exec test1 rping -s -a 1.1.1.1 -d -v&
ip netns add test2
ip link set veth-b netns test2
ip netns exec test2 ip l set veth-b up
ip netns exec test2 ip addr add 1.1.1.2/24 dev veth-b
ip netns exec test2 rdma link add rxe1 type rxe netdev veth-b
ip netns exec test2 ping -c 3 1.1.1.1
ip netns exec test2 rping -c -a 1.1.1.1 -d -v -C 3
ip netns ls
ip netns del test1
ip netns del test2
ip netns ls
```

3.Scenario: Multi net namespaces connected via veth



```
ip netns add one
ip netns add two
ip netns add three
ip link add veth-one-in type veth peer name veth-one-out
ip link add veth-two-in type veth peer name veth-two-out
ip link add veth-three-in type veth peer name veth-three-out
ip link set veth-one-in netns one
ip link set veth-two-in netns two
ip link set veth-three-in netns three
ip netns exec one ip addr add 10.0.0.10/24 dev veth-one-in
ip netns exec two ip addr add 10.0.0.20/24 dev veth-two-in
ip netns exec three ip addr add 10.0.0.30/24 dev veth-three-in
ip link add name virtual-bridge type bridge
ip link set veth-one-out master virtual-bridge
ip link set veth-two-out master virtual-bridge
ip link set veth-three-out master virtual-bridge
ip link set virtual-bridge up
ip link set veth-one-out up
ip link set veth-two-out up
ip link set veth-three-out up
ip netns exec one ip link set dev veth-one-in up
ip netns exec two ip link set dev veth-two-in up
ip netns exec three ip link set dev veth-three-in up
ip netns exec one ip link set dev lo up
ip netns exec two ip link set dev lo up
ip netns exec three ip link set dev lo up
ip netns exec one ping 10.0.0.10 -c1
ip netns exec one ping 10.0.0.20 -c1
ip netns exec one ping 10.0.0.30 -c1
ip netns exec one rdma link add rxe1 type rxe netdev veth-one-in
ip netns exec two rdma link add rxe2 type rxe netdev veth-two-in
ip netns exec three rdma link add rxe3 type rxe netdev veth-three-in
rdma link
ip netns exec one rping -s -a 10.0.0.10&
ip netns exec two rping -c -a 10.0.0.10 -d -v -C 3
ip netns exec one rping -s -a 10.0.0.10&
ip netns exec three rping -c -a 10.0.0.10 -d -v -C 3
ip netns del one
ip netns del two
ip netns del three
```

Thanks for your attention!

- Questions